

组成原理课程第一次实报告

实验名称：数据运算：定点加法

学号：2312331 姓名：王一诺 班次：张金老师

一、实验目的

1. 熟悉 LS-CPU-EXB-002 实验箱和软件平台。
2. 掌握利用该实验箱各项功能开发组成原理和体系结构实验的方法。
3. 理解并掌握加法器的原理和设计。
4. 熟悉并运用 verilog 语言进行电路设计。
5. 为后续设计 cpu 的实验打下基础。

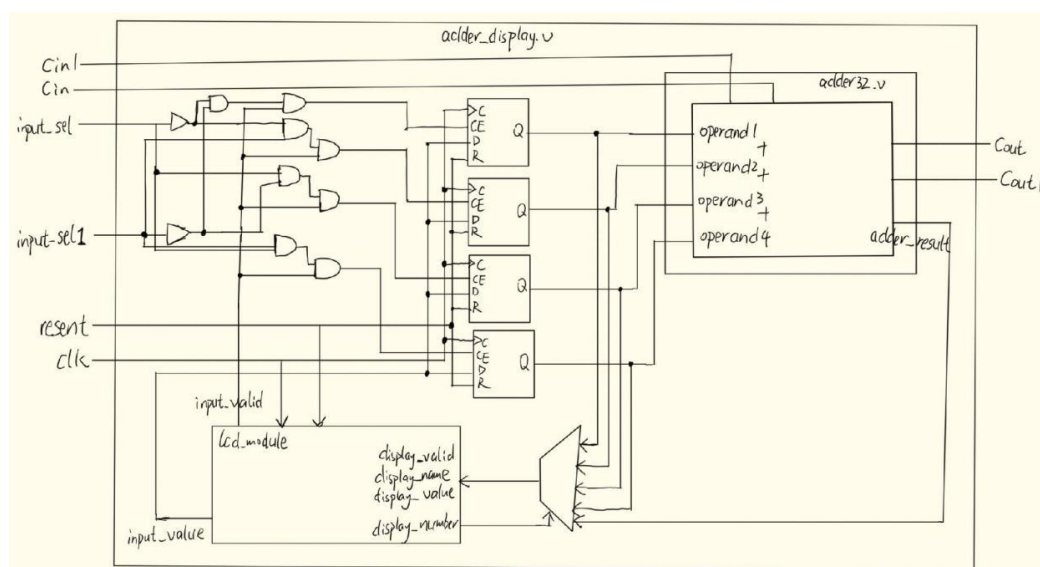
二、实验内容说明

根据第一次定点加法器实验进行扩展，实现四个数相加的加法器，注意以下问题：

1、四个数加法器的进位输入输出的位宽有调整；display 模块读数和显示数需要调整；约束文件需要调整。

2、实验报告中的原理图类似图 2.40 的形式，可以手绘；实验结果是实验箱照片及对应解释；实验报告中写明自己是怎么修改代码并实现功能的，不要全部贴代码。

三、实验原理图



从图中可以看到，外围模块 `adder_display` 内部调用了 `adder32` 和 `lcd_module` 模块。在外部除了时钟和复位信号外，还有 `cin` 和 `cin1` 两位进位输入、`input_sel` 和 `input_sel1` 通过拨码开关控制数字输入对象（四个数要输哪个数），以及 `cout`、`cout1` 两位高位进位输出。

四、实验步骤

(1) `adder32.v`

要把两数加法改为四数加法，首先要将 32 位加数扩充为 4 个。4 数相加可能会有两位高位进位输出，即除了向 33 位进位（记为 `cout`），还有可能向 34 位进位（记为 `cout1`）；同理，也就会有两位进位输入（`cin` 和 `cin1`）。`Cin1` 是在第二位上的进位输入，因此相加时需要 `x2` 再加上。修改如下：

```

module adder32(
    input [31:0] operand1,
    input [31:0] operand2,
    input [31:0] operand3,
    input [31:0] operand4,
    input cin,
    input cin1,
    output [31:0] result,
    output cout,
    output cout1
);
    assign (cout1,cout,result) = operand1 + operand2 + operand3 + operand4 + cin + 2*cin1;
endmodule

```

(2) adder_display.v

首先是进位输入、进位输出和加数数量的增加：

```

                                adder32 adder_module(
                                .operand1(adder_operand1),
                                .operand2(adder_operand2),
                                .operand3(adder_operand3),
                                .operand4(adder_operand4),
                                .cin      (adder_cin      ),
                                .cin1     (adder_cin1     ),
                                .result   (adder_result   ),
                                .cout     (adder_cout     ),
                                .cout1    (adder_cout1    )
                                );
    input sw_cin,
    input sw_cin1,
    output led_cout,
    output led_cout1,

    reg [31:0] adder_operand1;
    reg [31:0] adder_operand2;
    reg [31:0] adder_operand3;
    reg [31:0] adder_operand4;
    wire      adder_cin;
    wire      addr_cin1;
    wire [31:0] adder_result ;
    wire      adder_cout;
    wire      adder_cout1;

    assign adder_cin = sw_cin;
    assign adder_cin1=sw_cin1;
    assign led_cout  = adder_cout;
    assign led_cout1 =adder_cout1;

```

然后是加数输入选择逻辑的变化。原来两数加法时，用了 input_sel 来控制输入的是哪个数，即 sel=0 时显示器上输入的将是第一个加数，sel=1 时输入的将是第二个加数。为了实现四数的输入修改，用 sel 和 sel1 两个拨码开关来控制，00、01、10、11 将分别控制第 1、2、3、4 个加数的输入和修改。以选中加数 1 和加数 2 为例：

```

    always @(posedge clk)
    begin
        if (!resetrn)
        begin
            adder_operand1 <= 32'd0;
        end
        else if (input_valid && !input_sel&&!input_sel1)
        begin
            adder_operand1 <= input_value;
        end
    end
end

    always @(posedge clk)
    begin
        if (!resetrn)
        begin
            adder_operand2 <= 32'd0;
        end
        else if (input_valid && !input_sel&&input_sel1)
        begin
            adder_operand2 <= input_value;
        end
    end
end

```

关于触摸屏模块，将调用加法的个数从 2 增加到 4，即增加以下内容：

```

6'd3 :
begin
    display_valid <= 1'b1;
    display_name  <= "ADD_3";
    display_value <= adder_operand3;
end
6'd4 :
begin
    display_valid <= 1'b1;
    display_name  <= "ADD_4";
    display_value <= adder_operand4;
end

```

(3) Testbench 文件 (tb.v)

只需要将输入改成四个，随机数生成也相应地增加，进位也增加到两个即可：

```
reg [31:0] operand1;      adder32 uut (
reg [31:0] operand2;      .operand1(operand1),      operand1 = 0;
reg [31:0] operand3;      .operand2(operand2),      operand2 = 0;
reg [31:0] operand4;      .operand3(operand3),      operand3 = 0;
reg cin;                  .cin(cin),              operand4 = 0;
reg cin1;                  .cin1(cin1),             cin = 0;
wire [31:0] result;       .result(result),         cin1 = 0;
wire cout;                .cout(cout),
wire cout1;               .cout1(cout1)
                           );
                           always #10 operand3 = $random;
                           always #10 operand4 = $random;
                           always #10 cin = {$random} % 2;
                           always #10 cin1 = {$random} % 2;
```

(4) 约束文件 (adder.xdc)

只需更改控制加数选择的拨码开关，以及控制进位输入的拨码开关：

```
set_property PACKAGE_PIN AC23 [get_ports input_sel]
set_property PACKAGE_PIN AC22 [get_ports input_sel1]
set_property PACKAGE_PIN AD24 [get_ports sw_cin]
set_property PACKAGE_PIN AC21 [get_ports sw_cin1]
```

五、实验结果分析

1. 仿真验证

(仿真波形图右上角设置可以改为二进制数查看)

Name	Value	982, 810 ps	982, 811 ps	982, 812 ps	982, 813 ps	982, 814 ps	982, 815 ps	982, 816 ps	982, 817 ps	982, 818 ps	982, 819 ps
> operand1[31:0]	01011110001001010101000110111100										
> operand2[31:0]	10011000011110110000100000110000										
> operand3[31:0]	01100000001001110010110111000000										
> operand4[31:0]	00101000011101100110100101010000										
cin	1										
cin1	1										
> result[31:0]	01111111001111011111000011111111										
cout	1										
cout1	0										

01011110001001010101000110111100+

10011000011110110000100000110000+

01100000001001110010110111000000+

00101000011101100110100101010000+

(2*cin1+cin) 11= (如果写出完整的 34 位应当是:)

01011111100111101111100001111111

Result 只取后 32 位，即 01111111001111011111000011111111，

33 位上进位 cout=1，34 位上进位 cout1 为 0，测试正确。

Name	Value	999,992 ps	999,994 ps	999,996 ps	999,998 ps	1,000,000 ps
> operand1[31:0]	111110101111100110010111011110101	11111010111100110010111011110101				
> operand2[31:0]	0111101010000111101011111110101	0111101010000111101011111110101				
> operand3[31:0]	10101110111010101100110001011101	10101110111010101100110001011101				
> operand4[31:0]	11001010010010000001001010010100	11001010010010000001001010010100				
cin	0					
cin1	0					
> result[31:0]	11101110101011011011110111011011	11101110101011011011110111011011				
cout	0					
cout1	1					

```

11111010111100110010111011110101+
0111101010000111101011111110101+
10101110111010101100110001011101+
11001010010010000001001010010100+

```

00=（如果写出完整的 34 位应当是：）

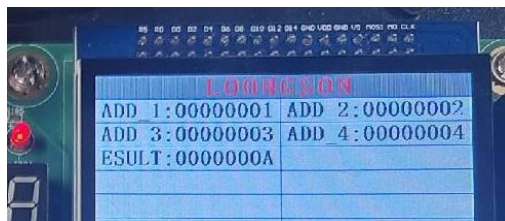
1011101110101011011011110111011011

Result 只取后 32 位，即 11101110101011011011110111011011，
33 位上进位 cout=0，34 位上进位 cout1 为 1，测试正确。

2. 实验箱验证

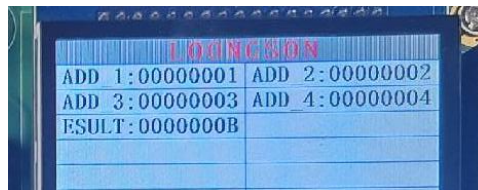
先依次设置四个加数为 1、2、3、4

当没有进位输入时（cin 和 cin1 拨到 0）：



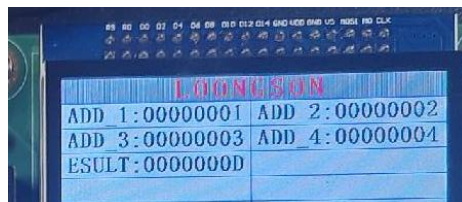
1+2+3+4=A（也就是 10），正确

将 cin 拨到 1：



1+2+3+4+1=B（也就是 11），正确

将 cin1 也拨到 1：



1+2+3+4+1*2+1=D（也就是 13），正确

六、总结感想

1. 学会了如何使用 vivado 创建项目、设计模块、设计仿真测试，对 verilog 语言有了一定程度的认知与掌握。

2. 学习了如何写约束文件，如何将程序烧制到实验箱上。测试与验证是做实验、设计模块与功能的重要一步，无论是仿真测试还是上实验箱检测，都应该认真设计与执行测试，分析与判断测试结果，甚至根据有问题的测试结果反思设计文件的错误与不足。

3. 认识了外围模块，此次实验的 `adder_display` 就是一个外围模块，它在模块内部调用了 `adder32.v` 以及 `ldc_module` 的内容。

4. 此次实验设计并不复杂，但也让我认识到设计的大致流程：从功能目标入手，分析逻辑关系或者表达式，尝试画出原理图，落实到程序代码，最后是测试验证。每一步都应认真对待，每一步也都能有所收获。